

Employee Task Management System

FullStack Web Application Development Hands-on Activity

Submitted By

Name: Janhavi Agrawal (002OD0744)

Role: Experienced Frontend Developer

Technology: FullStack Development (MERN Stack)

1. Project Overview

The Employee Task Management System is a role-based web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js).

The application allows employees to manage assigned tasks, managers to monitor and assign tasks, and administrators to oversee the entire system. The solution implements secure authentication, role-based access control, CRUD operations, database integration, and testing.

This project was developed as part of the FullStack Web Application Development Hands-on Activity.

2. Problem Statement

Organizations require a centralized platform for managing employee tasks and collaboration.

The system should provide:

- Secure user registration and login
- Role-based access control
- Task management capabilities
- Comment management
- Notification management
- Dashboard summaries
- Audit visibility for administrators

The application must support Admin, Manager, and Employee roles.

3. Selected Scenario

Scenario 1: Employee Task Management System

Roles

Admin

- Manage all users
- View all tasks
- View comments
- View notifications
- Access complete dashboard

Manager

- Create tasks
- Assign tasks
- Monitor task progress
- Access manager dashboard

Employee

- View assigned tasks
 - Update tasks
 - Add comments
 - Access employee dashboard
-

4. Technology Stack

Frontend

- React.js
- React Router DOM
- Axios
- JavaScript
- HTML5
- CSS3

Backend

- Node.js
- Express.js
- JWT Authentication
- Bcrypt.js
- Express Validator

Database

- MongoDB
- Mongoose ODM

Testing

- Jest
- Supertest

Version Control

- Git
 - GitHub
-

5. System Architecture

The application follows a three-tier architecture:

Presentation Layer

React Frontend

Components:

- Home
- Register
- Login
- Dashboard
- Tasks
- Comments
- Notifications

Business Logic Layer

Node.js + Express.js

Modules:

- Authentication
- Authorization
- Dashboard
- Task Management
- Comment Management
- Notification Management

Data Layer

MongoDB Database

Collections:

- Users
 - Tasks
 - Comments
 - Notifications
-

6. Database Design

User Collection

Fields:

- _id
 - name
 - email
 - password
 - role
 - createdAt
 - updatedAt
-

Task Collection

Fields:

- _id
- title
- description

- status
 - assignedTo
 - assignedBy
 - createdAt
 - updatedAt
-

Comment Collection

Fields:

- _id
 - task
 - comment
 - user
 - createdAt
 - updatedAt
-

Notification Collection

Fields:

- _id
 - user
 - message
 - isRead
 - createdAt
 - updatedAt
-

MongoDB Collections Screenshot

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total index size
comments	-	24.58 kB	0 B	0	0 B	1	24.58 kB
notifications	-	32.77 kB	0 B	0	0 B	1	32.77 kB
tasks	-	36.86 kB	353.00 B	2	176.00 B	1	36.86 kB
users	-	36.86 kB	1.31 kB	6	218.00 B	2	73.73 kB

7. API Design

Authentication APIs

Register User

POST /api/auth/register

Request:

```
{ "name": "John", "email": "john@example.com", "password": "password123",  
  "role": "Employee" }
```

Login User

POST /api/auth/login

Request:

```
{ "email": "john@example.com", "password": "password123" }
```

Logout User

POST /api/auth/logout

Dashboard APIs

GET /api/dashboard

Task APIs

POST /api/tasks

GET /api/tasks

GET /api/tasks/:id

PUT /api/tasks/:id

DELETE /api/tasks/:id

Comment APIs

POST /api/comments

GET /api/comments

PUT /api/comments/:id

DELETE /api/comments/:id

Notification APIs

POST /api/notifications

GET /api/notifications

PUT /api/notifications/:id/read

DELETE /api/notifications/:id

8. Authentication Implementation

Authentication is implemented using JWT (JSON Web Token).

Features:

- User Registration

- Password Hashing using Bcrypt
- User Login
- JWT Token Generation
- Protected APIs
- Logout Functionality

Security Measures:

- Passwords stored in encrypted format
 - JWT verification on protected routes
 - Middleware-based authorization
-

9. Authorization and RBAC

Role Based Access Control (RBAC) has been implemented.

Roles Supported

- Admin
- Manager
- Employee

Backend Authorization

Middleware:

- authMiddleware
- roleMiddleware

Frontend Authorization

Protected Route Component:

- ProtectedRoute.jsx

Unauthorized users cannot access secured pages.

10. Features Implemented

Screens

Home Page

Features:

- Application Introduction
- Public Access

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Employee Task Management System

Welcome to the Employee Task Management System.

Manage tasks, comments, notifications and dashboards based on user roles.

Registration Page

Features:

- Name
- Email
- Password
- Role Selection

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Register

Login Page

Features:

- Credential Validation
- JWT Generation

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Login

Dashboard

Features:

- Role Specific Dashboard
- Summary Statistics

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Dashboard

Welcome Janhavi

Role: Employee

Tasks Module

Features:

- Create Task
- View Tasks
- Update Task
- Delete Task

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Task Management

[Add Task](#)

Comments Module

Features:

- Create Comment
- View Comments

- Update Comment
- Delete Comment

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Comments

[Add Comment](#)

Notifications Module

Features:

- Create Notification
- View Notifications
- Mark As Read
- Delete Notification

Employee Task Management System

[Home](#) | [Login](#) | [Register](#) | [Dashboard](#) | [Tasks](#) | [Comments](#) | [Notifications](#)

[Home](#) | [Dashboard](#) | [Tasks](#) | [Logout](#)

Notifications

[Create Notification](#)

11. Error Handling

Backend Error Handling:

- 400 Validation Errors
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Server Errors

Frontend Error Handling:

- Validation Messages
 - Loading Indicators
 - Error Alerts
 - User Friendly Messages
-

12. Testing

Backend testing was implemented using Jest and Supertest.

Test Cases Covered:

- Root API
- Register User
- Login User
- Invalid Login
- Logout User

Results:

All tests passed successfully.

Test Execution Screenshot and Coverage Report Screenshot

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\JanhaviAgrawal\Documents\employee-task-management\backend> npm test

console.log
  LOGOUT RESPONSE:
    at Object.log (src/tests/app.test.js:75:13)

console.log
  { success: true, message: 'Logout successful' }
    at Object.log (src/tests/app.test.js:76:13)

console.log
  Closing Test DB...
    at Object.log (src/tests/setup.js:13:11)

PASS src/tests/app.test.js
  Root API
    ✓ GET / should return success message (17 ms)
  Auth API
    ✓ REGISTER user (316 ms)
    ✓ LOGIN user (161 ms)
    ✓ LOGIN wrong password (127 ms)
    ✓ LOGOUT user (61 ms)

-----
File                                % Stats % Branch % Funcs % Lines Uncovered Line #s
-----
All files                          70.37    22.5    32    70.37
src                                100    100    100    100
  app.js                           100    100    100    100
  src/config                        63.63    50    100    63.63  6,13-15
  db.js                            63.63    50    100    63.63
src/controllers                    47.36    17.85    17.64    47.36
  authController.js               84.84    62.5    100    84.84  12,23,47,61,88
  commentController.js           33.33    0    0    33.33  4-18,17-21,38-47,55-66
  dashboardController.js         33.33    0    0    33.33  7-44
  notificationController.js       33.33    0    0    33.33  4-9,16-19,28-45,53-64
  taskController.js              29.62    0    0    29.62  5-19,26-38,38-47,54-69,76-87
src/middlewares                   68.18    30    50    68.18
  authMiddleware.js              83.33    75    100    83.33  23-29
  errorHandler.js               50    0    0    50    7-9
  roleMiddleware.js              50    0    50    50    3-10
src/models                        100    100    100    100
  Comment.js                     100    100    100    100
  Notification.js                100    100    100    100
  Task.js                        100    100    100    100
  User.js                        100    100    100    100
src/routes                        98.03    100    100    98.03
  authRoutes.js                  100    100    100    100
  commentRoutes.js               100    100    100    100
  dashboardRoutes.js             100    100    100    100
  notificationRoutes.js          100    100    100    100
  taskRoutes.js                  100    100    100    100
  userRoutes.js                  83.33    100    0    83.33  11
src/utils                         100    100    100    100
  generateToken.js               100    100    100    100
src/validators                   100    100    100    100
  authValidator.js               100    100    100    100
  taskValidator.js               100    100    100    100

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:  0 total
Time:        2.483 s, estimated 3 s
Ran all test suites.
PS C:\Users\JanhaviAgrawal\Documents\employee-task-management\backend>
```

```
PS C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend> npm test

> frontend@0.0.0 test
> vitest

DEV v1.6.1 C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend
stderr | VirtualConsole, anonymous> (C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend\node_modules\jsdom\lib\jsdom\virtual-console.js:29:45)
Error: AggregateError
    at Object.dispatchError (C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend\node_modules\jsdom\lib\jsdom\living\xhr\xhr-utils.js:63:19)
    at EventEmitter.<anonymous> (C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend\node_modules\jsdom\lib\jsdom\living\xhr\xhr-utils.js:63:19)
    at EventEmitter.emit (node:events:530:35)
    at Request.<anonymous> (C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend\node_modules\jsdom\lib\jsdom\living\xhr\xhr-utils.js:404:14)
    at Request.emit (node:events:518:28)
    at ClientRequest.<anonymous> (C:\Users\JanhaviAgrawal\Documents\employee-task-management\frontend\node_modules\jsdom\lib\jsdom\living\helpers\http-request.js:127:14)
    at ClientRequest.emit (node:events:518:28)
    at Socket.socketErrorListener (node:http_client:500:9)
    at Socket.emit (node:events:518:28)
    at emitErrorNT (node:internal/streams/destroy:169:8) undefined
    ✓ src/tests/Home.test.jsx (1)
    ✓ src/tests/Register.test.jsx (1)
    ✓ src/tests/Dashboard.test.jsx (1)
    ✓ src/tests/Tasks.test.jsx (1)
    ✓ src/tests/Login.test.jsx (1)

Test Files 5 passed (5)
Tests      5 passed (5)
Start at   12:19:38
Duration   1.55s (transform 165ms, setup 650ms, collect 1.08s, tests 357ms, environment 3.55s, prepare 807ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

13. Challenges Faced

1. MongoDB connection issues during Jest testing.
 2. JWT authentication validation failures.
 3. Protected route implementation.
 4. CRUD synchronization between frontend and backend.
 5. Role-based dashboard rendering.
 6. Loading state implementation across modules.
-

14. Learnings

Through this project, the following skills were strengthened:

- React Development
 - REST API Development
 - Express Middleware
 - MongoDB Integration
 - JWT Authentication
 - Role-Based Authorization
 - CRUD Operations
 - Jest Testing
 - Error Handling
 - FullStack Application Architecture
-

15. Conclusion

The Employee Task Management System was successfully developed using the MERN stack. All mandatory requirements specified in the assignment were implemented, including authentication, authorization, role-based access control, CRUD operations, database integration, testing, and dashboard functionality.

The project demonstrates end-to-end FullStack development skills and provides a scalable foundation for enterprise task management solutions.